

將 Vision API 與 Dialogflow 整合

來源：<https://codelabs.developers.google.com/codelabs/chatbots-dialogflow-visionAPI?hl=zh-tw>

步驟數：18

瞭解如何將 Vision API 與 Dialogflow 整合。

目錄

1. 事前準備
2. 架構總覽
3. 什麼是 Vision API ?
4. 建立 Dialogflow 代理程式
5. 設定出貨程序，與 Vision API 整合
6. 下載並執行前端應用程式
7. 設定本機環境
8. 設定服務帳戶
9. 設定要從應用程式呼叫的 Dialogflow detectIntent 端點
10. 建立 Cloud Storage 值區
11. 在前端應用程式中設定 Cloud Storage 值區
12. 在本機建構及執行應用程式
13. 將應用程式部署至 App Engine 標準環境
14. 測試前端應用程式
15. 正式版
16. 測試您的聊天機器人
17. 清除所用資源
18. 恭喜

步驟 1 · 約 10 分鐘

1. 事前準備

在本程式碼研究室中，您將整合 Vision API 和 Dialogflow，根據使用者提供的圖片輸入內容，提供豐富且動態的機器學習回應。您將建立聊天機器人應用程式，以圖片做為輸入內容，在 Vision API 中處理圖片，然後向使用者傳回識別出的地標。舉例來說，如果使用者上傳泰姬瑪哈陵的圖片，聊天機器人就會傳回「泰姬瑪哈陵」做為回覆。

這項功能相當實用，因為您可以分析圖片中的項目，並根據取得的資訊採取行動。您也可以建立退款處理系統，協助使用者上傳收據、從收據中擷取購買日期，並在日期符合條件時處理退款。

請參閱下列對話範例：

使用者：Hi

聊天機器人：你好！上傳圖片即可探索地標

使用者：上傳含有泰姬瑪哈陵的圖片。

Chatbot：正在處理檔案，結果如下：泰姬瑪哈陵、泰姬瑪哈陵花園、泰姬瑪哈陵。

Dialogflow

Chat with Dialogflow

Hi

Hi! You can upload a picture to explore landmarks.



file is being processed, here are the results: Taj Mahal,Taj Mahal Garden,Taj Mahal

taj.jpeg

必要條件

繼續操作前，請先完成下列程式碼研究室：

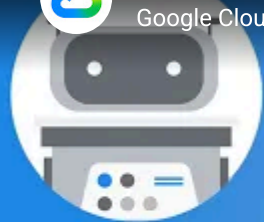
1. 使用 [Dialogflow](#) 建構預約排程器
2. 將 [Dialogflow](#) 聊天機器人與 [Actions on Google](#) 整合
3. 瞭解 [Dialogflow](#) 中的實體
4. 為 [Dialogflow](#) 應用程式建構前端 [Django](#) 用戶端

您也需要瞭解 Dialogflow 的基本概念和建構方式，這些內容可從「[使用 Dialogflow 建構聊天機器人](#)」路徑中的下列影片取得：



Deconstructing Chatbots - An Overview

Google Cloud Tech

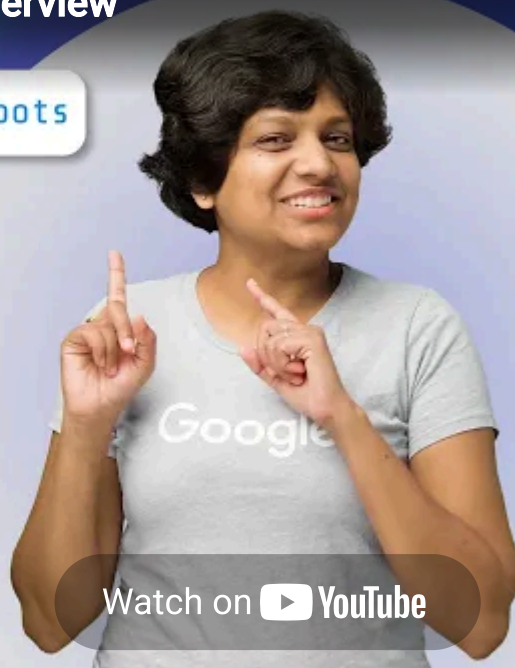


deconstructing chatbots

Deconstructing
Chatbots:
An overview



Watch on YouTube



1.



Getting Started with Dialogflow

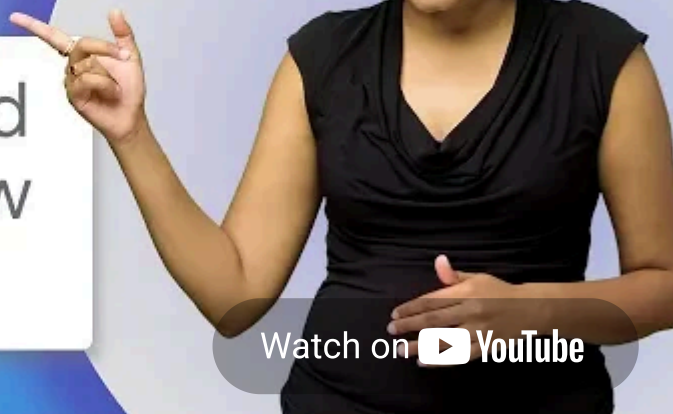
Google Cloud Tech

deconstructing chatbots

Getting Started
with Dialogflow

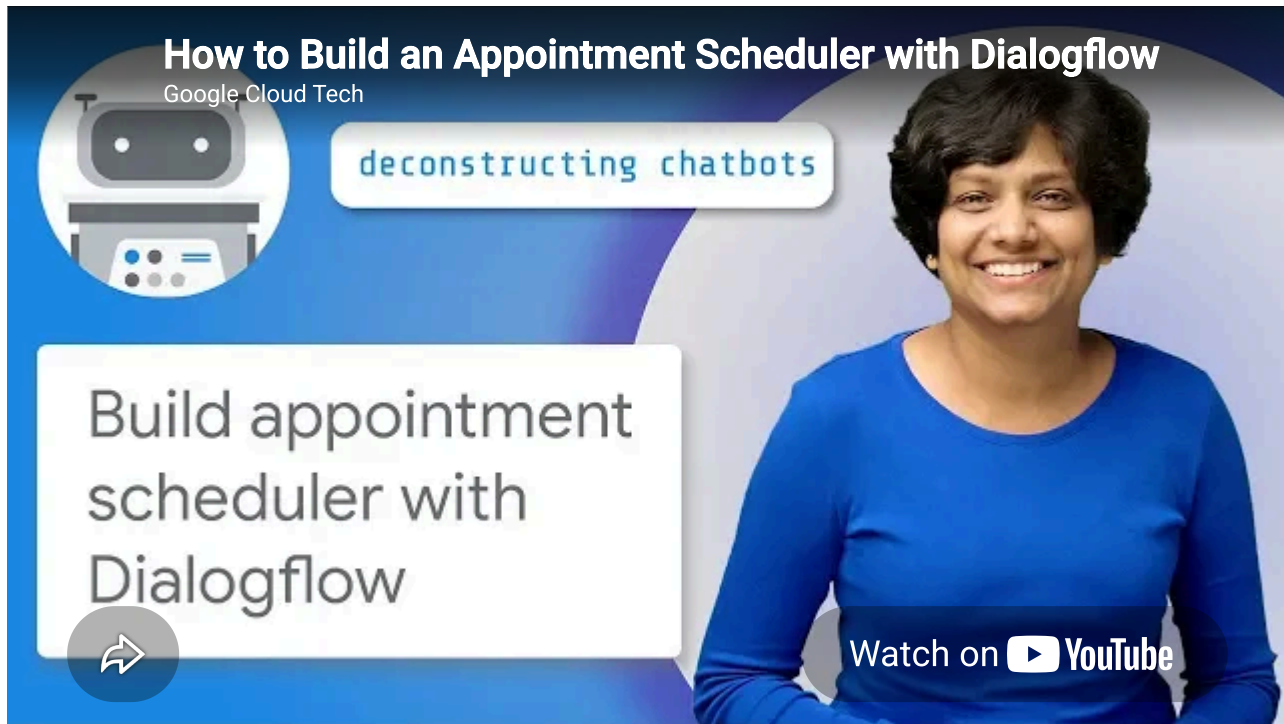


Watch on YouTube



2.

3.



This YouTube video thumbnail features a woman with short dark hair wearing a blue long-sleeved shirt. The background is a blue gradient with a circular graphic on the left containing a robot icon. Text elements include the title 'How to Build an Appointment Scheduler with Dialogflow', the channel name 'Google Cloud Tech', a 'deconstructing chatbots' tag, the video title 'Build appointment scheduler with Dialogflow', and a 'Watch on YouTube' button.

How to Build an Appointment Scheduler with Dialogflow

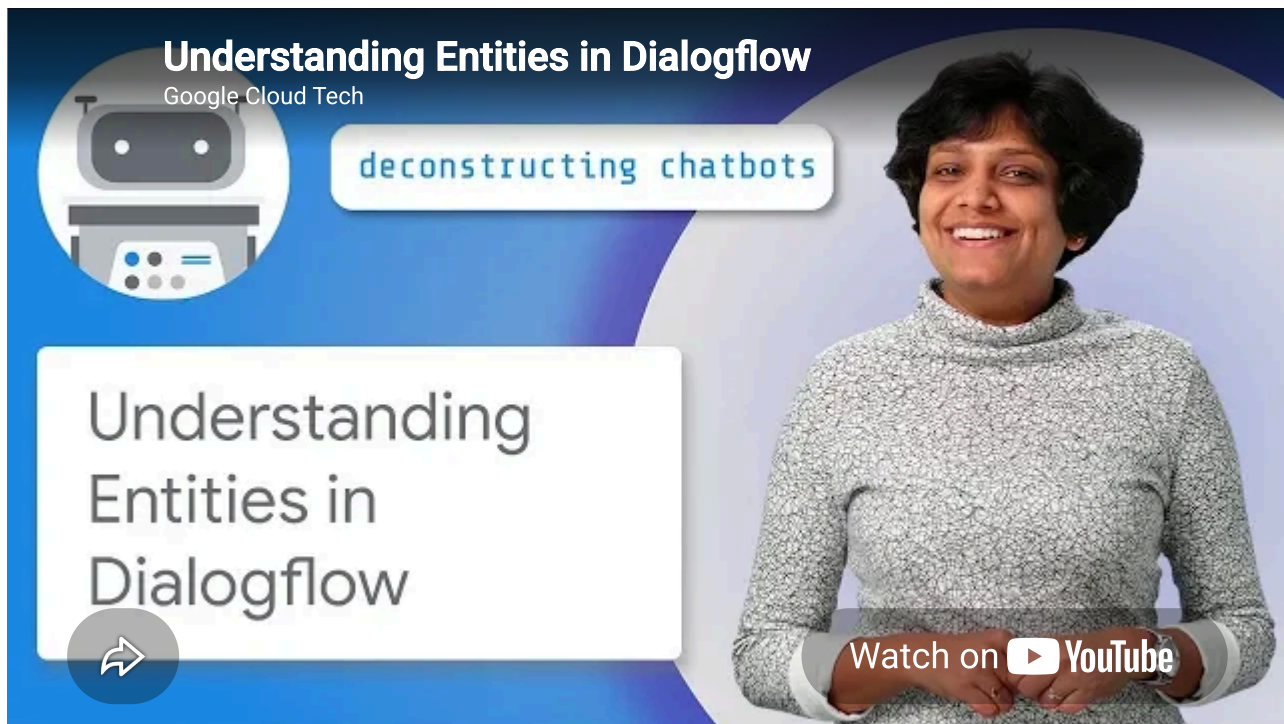
Google Cloud Tech

deconstructing chatbots

Build appointment scheduler with Dialogflow

Watch on  YouTube

4.



This YouTube video thumbnail features the same woman as the first thumbnail, but wearing a grey turtleneck sweater. The layout and text are identical to the first thumbnail, including the title 'Understanding Entities in Dialogflow'.

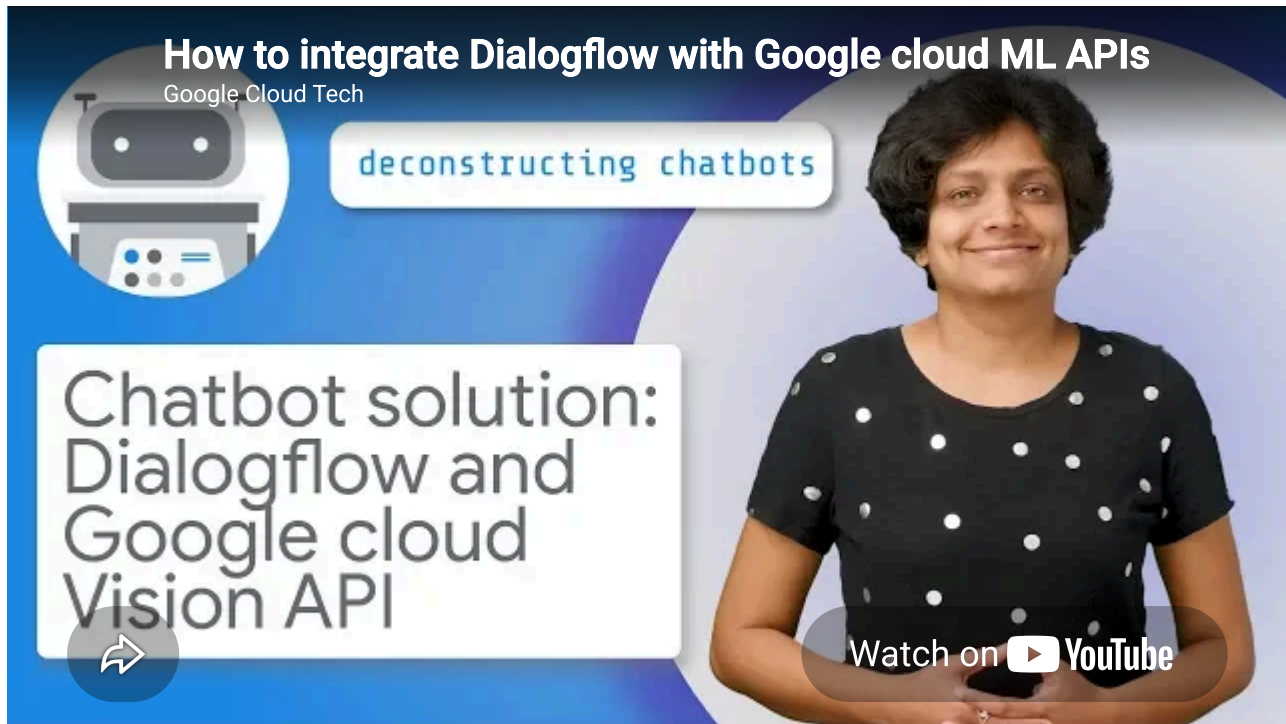
Understanding Entities in Dialogflow

Google Cloud Tech

deconstructing chatbots

Understanding Entities in Dialogflow

Watch on  YouTube



5.

課程內容

- 如何建立 Dialogflow 代理程式
- 如何更新 Dialogflow 代理程式來上傳檔案
- 如何透過 Dialogflow 實現設定 Vision API 連線
- 如何設定及執行 Dialogflow 的 Django 前端應用程式
- 如何將 Django 前端應用程式部署至 Google Cloud 的 App Engine
- 如何從自訂前端測試 Dialogflow 應用程式

建構項目

- 建立 Dialogflow 代理程式
- 實作 Django 前端，上傳檔案
- 實作 Dialogflow 實現功能，針對上傳的圖片呼叫 Vision API

軟硬體需求

- Python 的基本知識
 - 對 Dialogflow 有基本瞭解
 - 對 Vision API 有基本瞭解
-

步驟 2 · 約 3 分鐘

2. 架構總覽

您將使用自訂 Django 前端建立新的對話式服務，並擴充該服務以整合 Vision API。您將使用 Django 架構建構前端、在本機執行及測試，然後部署至 App Engine。前端看起來會像這樣：

Dialogflow

Chat with Dialogflow

Type something to begin...

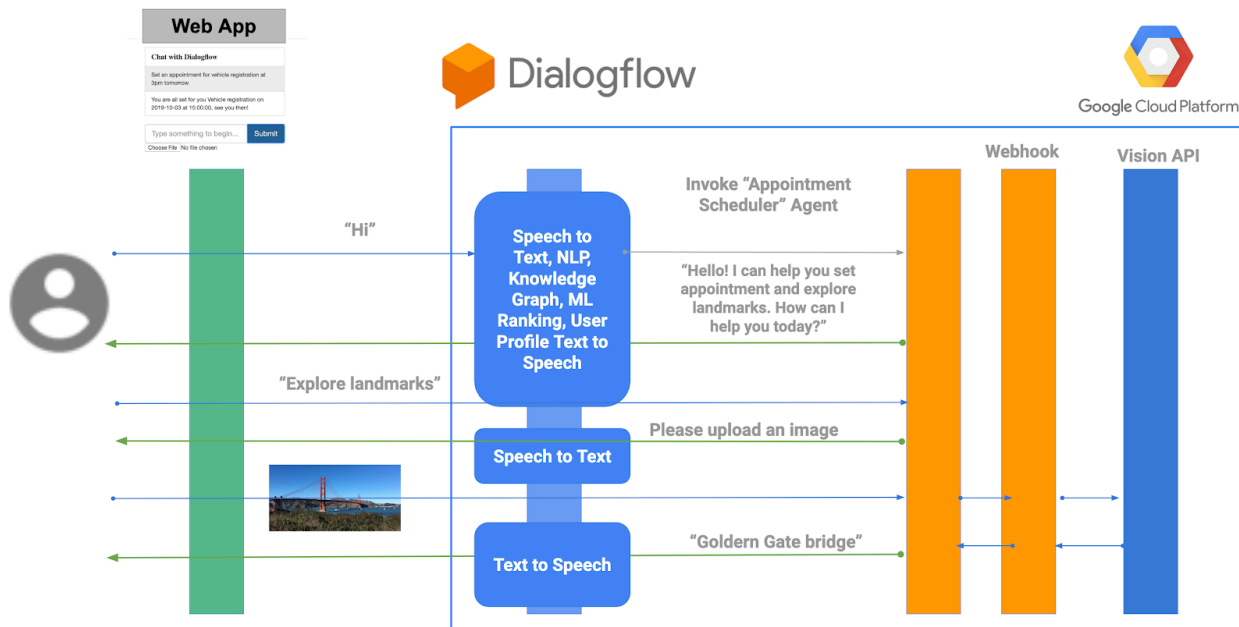
Submit

Choose File

No file chosen

要求流程如下圖所示：

1. 使用者會透過前端傳送要求。
2. 這會觸發對 Dialogflow detectIntent API 的呼叫，將使用者話語對應至正確意圖。
3. 偵測到探索地標意圖後，Dialogflow 執行要求會將要求傳送至 Vision API、接收回應，然後傳送給使用者。



整體架構如下所示。

Dialogflow

Chat with Dialogflow

Hi

Greetings! I can help you set appointment and explore landmarks.
How can I assist?

explore landmarks

Sure, please upload the image.

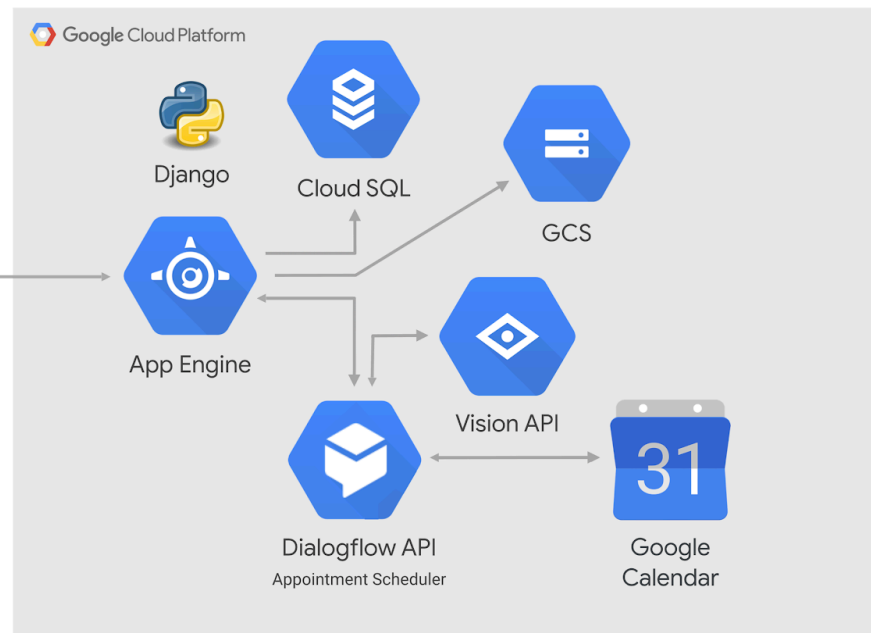


file is being processed, here are the results: Golden Gate Bridge,Golden Gate National Recreation Area,Golden Gate Bridge,Golden Gate Bridge,Golden Gate Bridge

Type something to begin...

Submit

Choose File | demo.jpg



步驟 3 · 約 2 分鐘

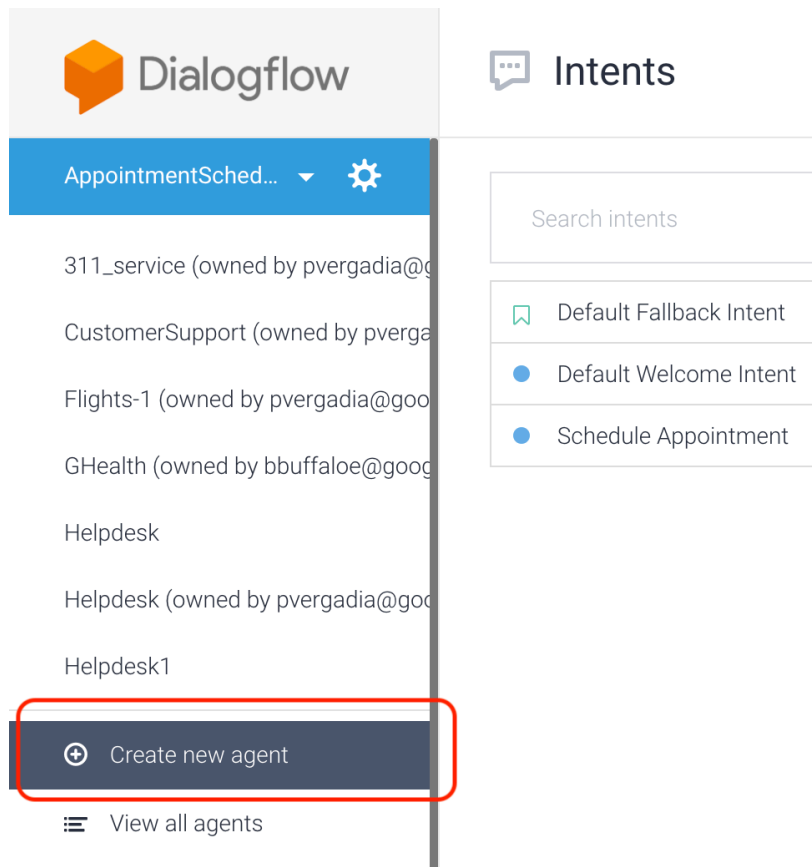
3. 什麼是 Vision API ?

Vision API 是預先訓練的 ML 模型，可從圖片中取得洞察資訊。這項服務可提供多種洞察資訊，包括為圖片加上標籤、偵測臉部和地標、光學字元辨識，以及為煽情露骨內容加上標記。詳情請參閱 [Vision AI](#)。

4. 建立 Dialogflow 代理程式

1. 前往 [Dialogflow 主控台](#)。
2. 登入即可(如果是首次使用，請使用電子郵件地址註冊。)
3. 接受條款及細則，即可進入管理中心。
4. 按一下

，捲動至底部，然後按一下「建立新代理程式」。



5. 輸入「VisionAPI」做為代理程式名稱。
6. 點選「建立」。

Dialogflow 會建立下列兩個預設意圖，做為代理程式的一部分：

1. 預設歡迎意圖會歡迎使用者。
2. 預設的回退意圖會擷取機器人無法理解的所有問題。

此時，您已擁有可向使用者問候的機器人，但需要更新機器人，讓使用者知道可以上傳圖片來探索地標。

更新預設歡迎意圖，通知使用者上傳圖片

1. 點選 [Default Welcome Intent] (預設的歡迎意圖)。
2. 依序前往「Responses」>「Default」>「Text or SSML Response」，然後輸入「Hi! 你可以上傳圖片來探索地標。」

• Default Welcome Intent

SAVE

Responses ?

DEFAULT +

Text Response

? ?

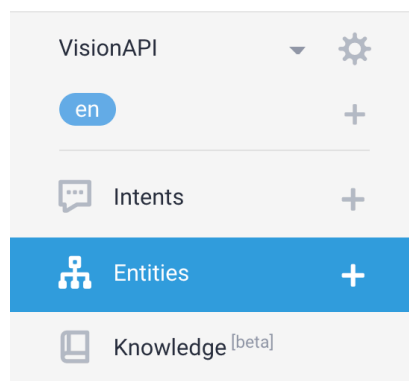
- 1 Hi! You can upload a picture to explore landmarks.
- 2 Hello! Please upload a picture to locate landmarks.
- 3 Good day! Let's explore landmarks. Please upload a picture.
- 4 Greetings! Upload a picture and I can help you locate a landmark
- 5 Enter a text response variant

ADD RESPONSES

☐ Set this intent as end of conversation ?

建立實體

1. 按一下「實體」。



2. 按一下「Create Entity」(建立實體)，將實體命名為「filename」，然後按一下「Save」(儲存)。

filename

SAVE

- ☒ Define synonyms ? ☐ Regexp entity ? ☒ Allow automated expansion
☒ Fuzzy matching ?

demo.jpeg	demo.jpeg
taj.jpg	taj.jpg
demo.jpg	demo.jpg
Click here to edit entry	

建立新意圖

1. 依序點選「意圖」 > 「建立意圖」。
2. 輸入「Explore uploaded image」做為「Intent name」(意圖名稱)。
3. 依序點選「訓練詞組」 > 「新增訓練詞組」，然後輸入「檔案是 demo.jpg」和「檔案是 taj.jpeg」做為使用者表達內容，並將 **@filename** 設為實體。

• Explore uploaded image

SAVE

Contexts ?

Events ?

Training phrases ?

Search training ph

” Add user expression

” file is demo.jpg

PARAMETER NAME	ENTITY	RESOLVED VALUE
filename	@filename	demo.jpg

” file is taj.jpeg

” file is eiffel.jpeg

- 依序點選「Responses」>「Add Response」>「Default」>「Text or SSML Response」。輸入「正在評估檔案」，然後按一下「新增回覆」。
- 依序點選「Fulfillment」(執行要求)>「Enable fulfillment」(啟用執行要求)，然後開啟「Enable webhook call for this intent」(為這個意圖啟用 Webhook 呼叫)。

Responses ?

DEFAULT +

Text Response

- Assessing file
- Enter a text response variant

ADD RESPONSES

☐ Set this intent as end of conversation ?

Fulfillment ?

☒ Enable webhook call for this intent

☐ Enable webhook call for slot filling

步驟 5 · 約 10 分鐘

5. 設定出貨程序，與 Vision API 整合

1. 按一下「履行」。
2. 啟用「Inline Editor」(內嵌編輯器)。

Inline Editor (Powered by Cloud Functions for Firebase)

ENABLED 

3. 使用下列程式碼更新 `index.js`，並將 `YOUR-BUCKET-NAME` 更新為 Cloud Storage 值區名稱。

```

'use strict';

const functions = require('firebase-functions');
const {google} = require('googleapis');
const {WebhookClient} = require('dialogflow-fulfillment');
const vision = require('@google-cloud/vision');

/**
 * TODO(developer): Uncomment the following lines before running the sample.
 */

const bucketName = 'YOUR-BUCKET-NAME';
const timeZone = 'America/Los_Angeles';
const timeZoneOffset = '-07:00';

exports.dialogflowFirebaseFulfillment = functions.https.onRequest((request, response) => {
  const agent = new WebhookClient({ request, response });
  console.log("Parameters", agent.parameters);

  function applyML(agent){
    const filename = agent.parameters.filename;
    console.log("filename is: ", filename);

    // call vision API to detect text
    return callVisionApi(agent, bucketName, filename).then(result => {
      console.log(`result is ${result}`);
      agent.add(`file is being processed, here are the results:  ${result}`);
      //agent.add(`file is being processed ${result}`);
    }).catch((error)=> {
      agent.add(`error occurred at apply ml function` + error);
    });
  }

  let intentMap = new Map();
  intentMap.set('Explore uploaded image', applyML);
  agent.handleRequest(intentMap);
});

async function callVisionApi(agent, bucketName, fileName){
  // [START vision_text_detection_gcs]
  // Imports the Google Cloud client libraries
  // Creates a client

  const client = new vision.ImageAnnotatorClient();
  try {
    // Performs text detection on the gcs file
    const [result] = await client.landmarkDetection(`gs://${bucketName}/${fileName}`);
    const detections = result.landmarkAnnotations;
    var detected = [];
    detections.forEach(text => {
      console.log(text.description);
      detected.push(text.description);
    });
  }

```

```
        return detected;
    }
    catch(error) {
        console.log('fetch failed', error);
        return [];
    }
}
```

4. 將下列內容貼到 `package.json` 中，取代原有內容。

```
{
  "name": "dialogflowFirebaseFulfillment",
  "description": "Dialogflow fulfillment for the bike shop sample",
  "version": "0.0.1",
  "private": true,
  "license": "Apache Version 2.0",
  "author": "Google Inc.",
  "engines": {
    "node": "6"
  },
  "scripts": {
    "lint": "semistandard --fix \"**/*.js\"",
    "start": "firebase deploy --only functions",
    "deploy": "firebase deploy --only functions"
  },
  "dependencies": {
    "firebase-functions": "2.0.2",
    "firebase-admin": "^5.13.1",
    "actions-on-google": "2.2.0",
    "googleapis": "^27.0.0",
    "dialogflow-fulfillment": "^0.6.1",
    "@google-cloud/bigquery": "^1.3.0",
    "@google-cloud/storage": "^2.0.0",
    "@google-cloud/vision": "^0.25.0"
  }
}
```

5. 按一下 [儲存]。

步驟 6 · 約 2 分鐘

6. 下載並執行前端應用程式

1. 將這個存放區複製到本機電腦：

```
https://github.com/priyankavergadia/visionapi-dialogflow.git
```

2. 切換至包含程式碼的目錄。您也可以下載 zip 格式的範例，然後解壓縮該檔案。

```
cd visionapi-dialogflow
```

步驟 7 · 約 3 分鐘

7. 設定本機環境

部署作業完成後，應用程式會使用 App Engine 標準環境內建的 Cloud SQL Proxy 與您的 Cloud SQL 執行個體進行通訊。不過，如要在本機測試應用程式，您就必須在開發環境安裝並使用 Cloud SQL Proxy 本機副本。詳情請參閱「[關於 Cloud SQL Proxy](#)」。

如要在 Cloud SQL 執行個體上執行基本的管理工作，您可以使用 MySQL 適用的 Cloud SQL 用戶端。

注意：您必須使用 gcloud 指令列工具進行驗證，才能從本機電腦使用 Proxy 連線

安裝 Cloud SQL Proxy

使用下列指令下載並安裝 Cloud SQL Proxy。Cloud SQL Proxy 用於在本機執行時連線至 Cloud SQL 執行個體。

下載 Proxy：

```
curl -o cloud_sql_proxy https://dl.google.com/cloudsql/cloud_sql_proxy.darwin.amd64
```

將 Proxy 設為可執行。

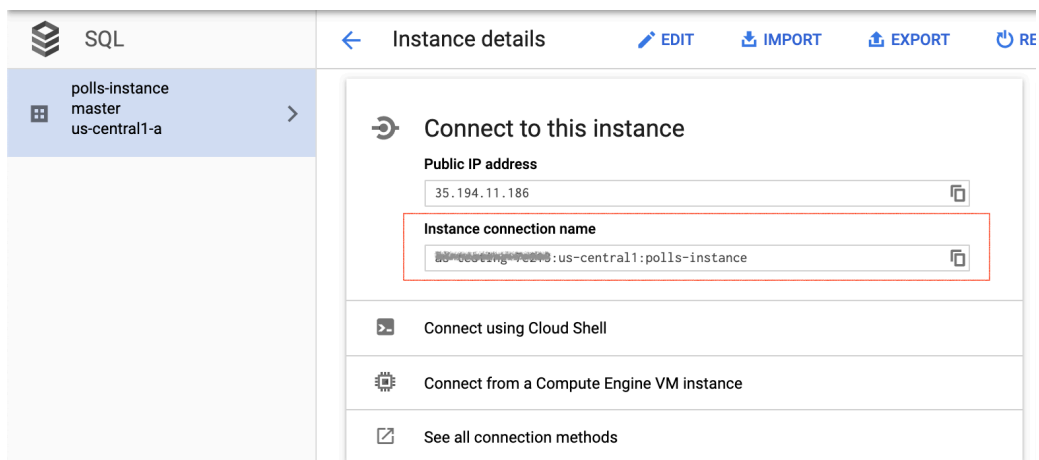
```
chmod +x cloud_sql_proxy
```

建立 Cloud SQL 執行個體

1. 建立 MySQL 適用的 Cloud SQL 第二代執行個體。輸入「polls-instance」或類似名稱。執行個體可能需要幾分鐘的時間才能準備就緒。準備就緒後，應該會顯示在執行個體清單中。
2. 現在請使用 gcloud 指令列工具執行下列指令，其中 [YOUR_INSTANCE_NAME] 是您 Cloud SQL 執行個體的名稱。記下系統顯示的 connectionName 值，以供下個步驟使用。顯示格式為 [PROJECT_NAME]:[REGION_NAME]:[INSTANCE_NAME]。

```
gcloud sql instances describe [YOUR_INSTANCE_NAME]
```

或者，您也可以直接在控制台中點選執行個體，取得「執行個體連線名稱」。



初始化 Cloud SQL 執行個體

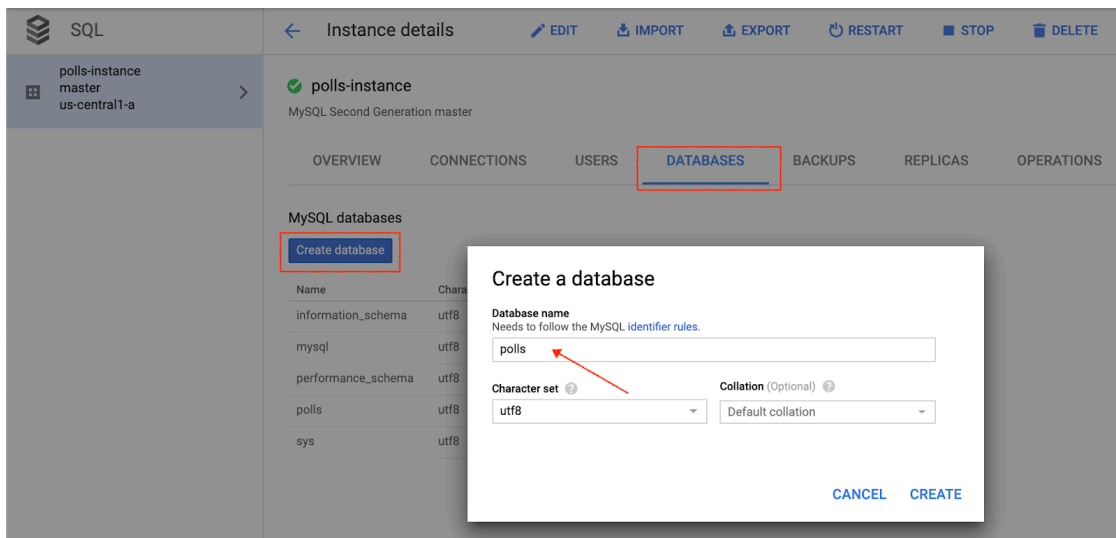
使用上一節的 `connectionName` 啟動 Cloud SQL Proxy。

```
./cloud_sql_proxy -instances="[YOUR_INSTANCE_CONNECTION_NAME]"=tcp:3306
```

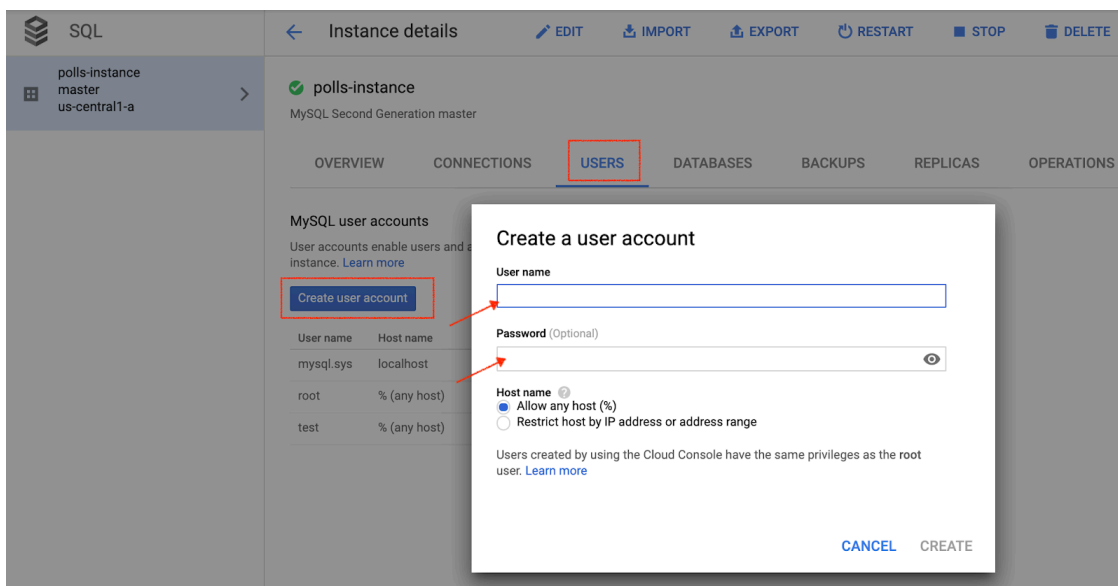
將 `[YOUR_INSTANCE_CONNECTION_NAME]` 換成您在前一節記下的值。這會將本機電腦連線至 Cloud SQL 執行個體，以利進行本機測試。在本機測試應用程式期間，請讓 Cloud SQL Proxy 全程保持運作。

接著，建立新的 Cloud SQL 使用者與資料庫。

1. 使用 Google Cloud 控制台，為名為 **polls-instance** 的 Cloud SQL 執行個體建立新資料庫。例如，你可以輸入「polls」做為名稱。



2. 使用 Cloud Console 為名為 **polls-instance** 的 Cloud SQL 執行個體建立新使用者。



進行資料庫設定

1. 開啟 `mysite/settings-changeme.py` 進行編輯。
2. 將檔案重新命名為 `setting.py`。

3. 請將 `[YOUR-USERNAME]` 和 `[YOUR-PASSWORD]` 兩處更換為您在上節中建立的資料庫使用者名稱和密碼。這有助於針對 App Engine 部署及本機測試設定連線至資料庫。
4. 在第 `'HOST': 'cloudsql/[PROJECT_NAME]:[REGION_NAME]:[INSTANCE_NAME]'` 行中，將 `[PROJECT_NAME]:[REGION_NAME]:[INSTANCE_NAME]` 替換為您在上一個部分取得的執行個體名稱。
5. 執行下列指令，並複製輸出的 `connectionName` 值用於下個步驟。

```
gcloud sql instances describe [YOUR_INSTANCE_NAME]
```

6. 將 `[YOUR-CONNECTION-NAME]` 改為您在上一步記下的值
7. 將 `[YOUR-DATABASE]` 替換為您在上一個章節中選擇的名稱。

```
# [START db_setup]
if os.getenv('GAE_APPLICATION', None):
    # Running on production App Engine, so connect to Google Cloud SQL using
    # the unix socket at /cloudsql/<your-cloudsql-connection string>
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.mysql',
            'HOST': '/cloudsql/[PROJECT_NAME]:[REGION_NAME]:[INSTANCE_NAME]',
            'USER': '[YOUR-USERNAME]',
            'PASSWORD': '[YOUR-PASSWORD]',
            'NAME': '[YOUR-DATABASE]',
        }
    }
else:
    # Running locally so connect to either a local MySQL instance or connect to
    # Cloud SQL via the proxy. To start the proxy via command line:
    #     $ cloud_sql_proxy -instances=[INSTANCE_CONNECTION_NAME]=tcp:3306
    # See https://cloud.google.com/sql/docs/mysql-connect-proxy
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.mysql',
            'HOST': '127.0.0.1',
            'PORT': '3306',
            'NAME': '[YOUR-DATABASE]',
            'USER': '[YOUR-USERNAME]',
            'PASSWORD': '[YOUR-PASSWORD]'
        }
    }
# [END db_setup]
```

8. 關閉並儲存 `settings.py`。

步驟 8 · 約 2 分鐘

8. 設定服務帳戶

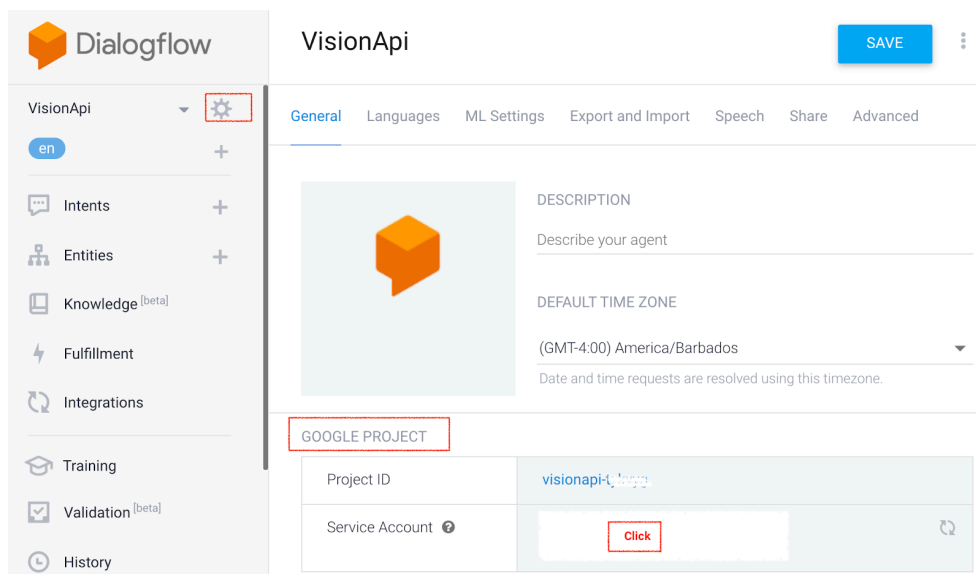
1. 在 Dialogflow 主控台中，按一下



- 。在「一般」分頁中，依序前往「Google 專案」>「專案 ID」，然後按一下「Google Cloud」



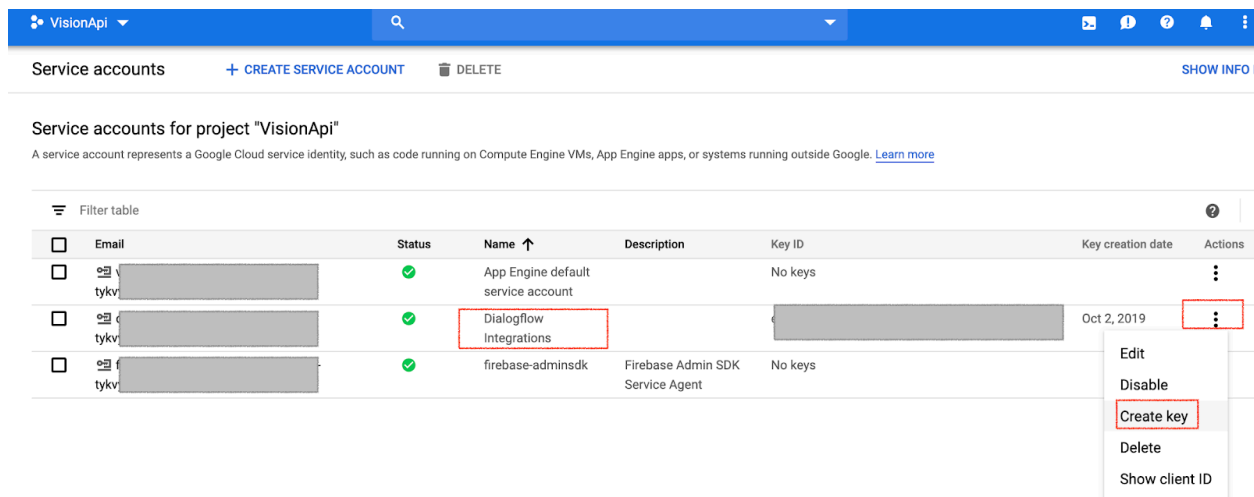
開啟 Cloud 控制台。



2. 依序點選「導覽選單 ≡」>「IAM 與管理」>「服務帳戶」，然後點選「Dialogflow integrations」(Dialogflow 整合) 旁的



，再點選「建立金鑰」。



3. 系統會將 JSON 檔案下載至您的電腦，您需要在後續設定章節中使用該檔案。

步驟 9 · 約 1 分鐘

9. 設定要從應用程式呼叫的 Dialogflow detectIntent 端點

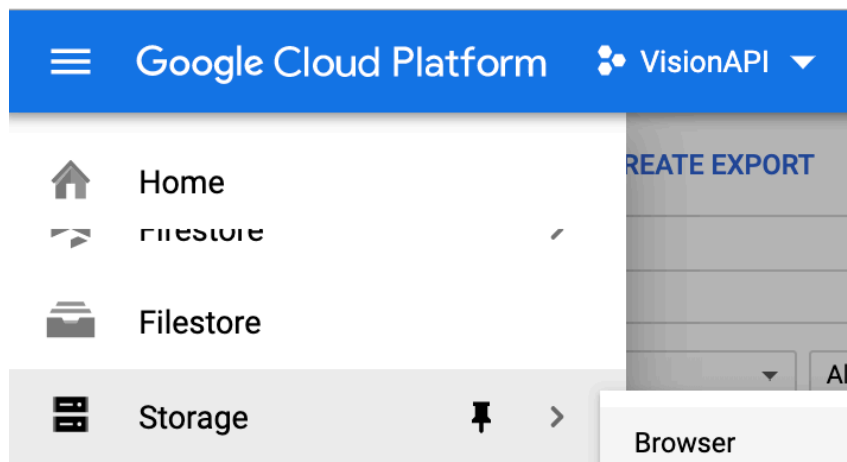
1. 在 chat 資料夾中，將 `key-sample.json` 替換成您的憑證 JSON 檔案，並命名為 `key.json`。
 2. 在聊天資料夾的 `views.py` 中，將 `GOOGLE_PROJECT_ID = "<YOUR_PROJECT_ID>"` 變更為您的專案 ID。
-

步驟 10 · 約 3 分鐘

10. 建立 Cloud Storage 值區

為前端靜態物件建立 Cloud Storage bucket

1. 在 Cloud 控制台中，依序點選「導覽選單」圖示 ≡ > 「儲存空間」。



2. 按一下「建立值區」。
3. 提供全域不重複的名稱。

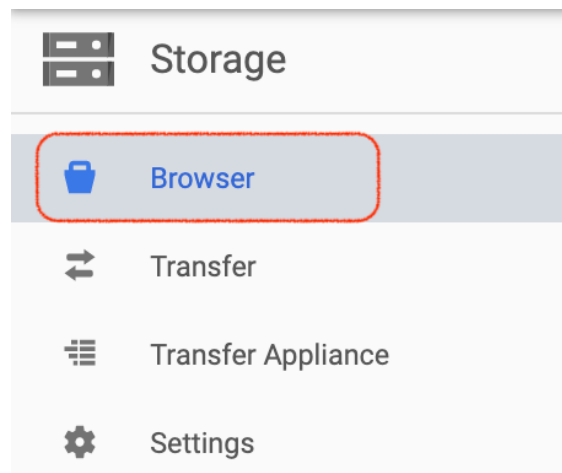
A screenshot of the 'Create a bucket' form in the Google Cloud Storage console. The form has a title 'Create a bucket' with a back arrow. Below the title, there's a section 'Name your bucket' with a bullet point. The text says 'Pick a globally unique, permanent name. [Naming guidelines](#)'. There is a text input field containing 'sample-test-name'. Below the input field, a tip says 'Tip: Don't include any sensitive information'. At the bottom of the section, there is a 'CONTINUE' button.

4. 選擇資料的儲存位置。選取「區域」，然後選擇最符合需求的地區。
5. 選擇「Standard」做為預設儲存空間級別。

6. 選擇「在值區層級統一設定權限 (僅值區政策)」，然後按一下「繼續」建立值區。

- **Choose how to control access to objects**

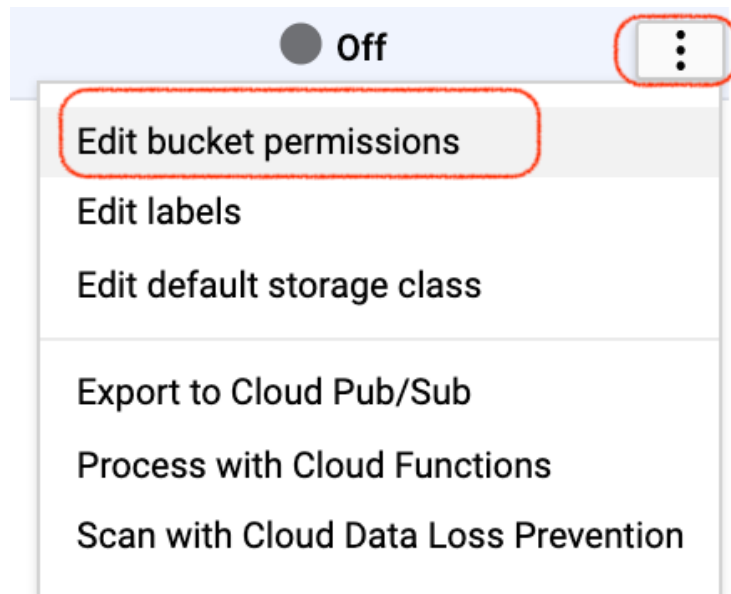
7. 建立 bucket 後，依序點選「導覽選單」圖示 ≡ > 「Storage」 > 「瀏覽器」，然後找出您建立的 bucket。



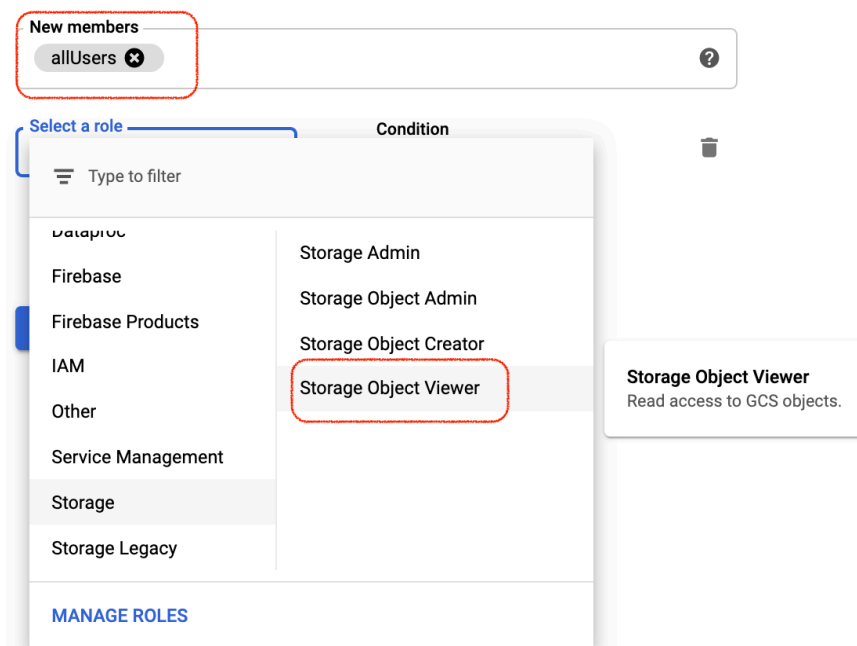
8. 按一下對應 bucket 旁邊的



，然後按一下「編輯 bucket 權限」。



9. 按一下「新增成員」，然後點按「新成員」，輸入「allUsers」，然後依序點按「請選擇角色」>「Storage 物件檢視者」。這會為 **allUsers** 提供靜態前端檔案的檢視權限。這並非檔案的理想安全性設定，但適用於本程式碼研究室。



建立 Cloud Storage bucket，存放使用者上傳的圖片

請按照相同的操作說明建立另一個 bucket，用來上傳使用者圖片。再次將權限設為「allUsers」，但選取「Storage 物件建立者」和「Storage 物件檢視者」做為角色。

11. 在前端應用程式中設定 Cloud Storage 值區

在 settings.py 中設定 Cloud Storage 值區

1. 開啟 `mysite/setting.py` 。
2. 找出 `GCS_BUCKET` 變數，並將 '`<YOUR-GCS-BUCKET-NAME>`' 替換為您的 Cloud Storage 靜態 bucket 。
3. 找出 `GS_MEDIA_BUCKET_NAME` 變數，並將 '`<YOUR-GCS-BUCKET-NAME-MEDIA>`' 換成圖片的 Cloud Storage 值區名稱。
4. 找出 `GS_STATIC_BUCKET_NAME` 變數，並將 '`<YOUR-GCS-BUCKET-NAME-STATIC>`' 換成靜態檔案的 Cloud Storage 值區名稱。
5. 儲存檔案。

```
GCS_BUCKET = '<YOUR-GCS-BUCKET-NAME>'
GS_MEDIA_BUCKET_NAME = '<YOUR-GCS-BUCKET-NAME-MEDIA>'
GS_STATIC_BUCKET_NAME = '<YOUR-GCS-BUCKET-NAME-STATIC>'
```

在 home.html 中設定 Cloud Storage bucket

- 開啟即時通訊資料夾，然後開啟 `templates`，並將 `home-changeme.html` 重新命名為 `home.html` 。
 - 找出 `<YOUR-GCS-BUCKET-NAME-MEDIA>`，並將其替換為您要儲存使用者上傳檔案的值區名稱。這樣一來，您就無法將使用者上傳的檔案儲存在前端，並將靜態資產保留在 Cloud Storage 值區中。Vision API 會呼叫 Cloud Storage bucket 來擷取檔案並進行預測。
-

12. 在本機建構及執行應用程式

如要在本機電腦執行 Django 應用程式，就需要設定 Python 開發環境，包括 Python、pip 及 virtualenv。如需操作說明，請參閱「[設定 Python 開發環境](#)」。

1. 建立獨立的 Python 環境，並安裝依附元件。

```
virtualenv env
source env/bin/activate
pip install -r requirements.txt
```

2. 執行 Django 遷移作業以設定模型。

```
python3 manage.py makemigrations
python3 manage.py makemigrations polls
python3 manage.py migrate
```

3. 啟動本機網路伺服器。

```
python3 manage.py runserver
```

4. 在網路瀏覽器中，前往 <http://localhost:8000/>。您應該會看到如下所示的簡單網頁：

Dialogflow

Chat with Dialogflow

Type something to begin...

Submit

Choose File No file chosen

範例應用程式頁面是由在您的電腦上執行的 Django 網路伺服器提供。準備好繼續時，請按下 `Control+C` 鍵 (Macintosh 上的 `Command+C` 鍵) 停止本機網路伺服器。

使用 Django 管理控制台

1. 建立超級使用者。

```
python3 manage.py createsuperuser
```

2. 啟動本機網路伺服器。

```
python3 manage.py runserver
```

3. 在網路瀏覽器中前往 <http://localhost:8000/admin/>。如要登入管理員網站，請輸入您在執行 `createsuperuser` 時建立的使用者名稱和密碼。
-

步驟 13 · 約 3 分鐘

13. 將應用程式部署至 App Engine 標準環境

執行下列指令，將所有靜態內容收集至單一資料夾。這項指令會將所有應用程式的靜態檔案，移至 `settings.py` 之中 `STATIC_ROOT` 指定的資料夾：

```
python3 manage.py collectstatic
```

從 `app.yaml` 檔案所在的應用程式目錄執行下列指令，以上傳應用程式：

```
gcloud app deploy
```

等候更新完成通知訊息。

步驟 14 · 約 1 分鐘

14. 測試前端應用程式

在網路瀏覽器中前往 https://<your_project_id>.appspot.com

這次您的要求是由在 App Engine 標準環境執行的網路伺服器提供。

`app deploy` 指令會部署 `app.yaml` 中描述的應用程式，並將新部署的版本設為預設版本，將其用來處理所有新流量。

步驟 15 · 約 1 分鐘

15. 正式版

準備好在實際工作環境提供內容時，請在 `mysite/settings.py` 中將 `DEBUG` 變數變更為 `False`。

16. 測試您的聊天機器人

您可以在模擬器中測試聊天機器人，或使用先前建立的網站或 Google Home 整合服務。

- 1. 使用者：「你好」
- 2. 聊天機器人：「你好！你可以上傳圖片來探索地標。」
- 3. 使用者上傳圖片。

下載這張圖片並命名為 `demo.jpg`，然後使用。



- 4. 聊天機器人：「檔案正在處理中，結果如下：金門大橋、金門國家遊樂區、金門大橋、金門大橋、金門大橋。」

整體而言，應該會類似這樣：

Dialogflow

Chat with Dialogflow

hi

Hi! You can upload a picture to explore landmarks.



file is being processed, here are the results: Golden Gate Bridge,Golden Gate National Recreation Area,Golden Gate Bridge,Golden Gate Bridge,Golden Gate Bridge

Type something to begin...

Submit

Choose File

demo.jpg

17. 清除所用資源

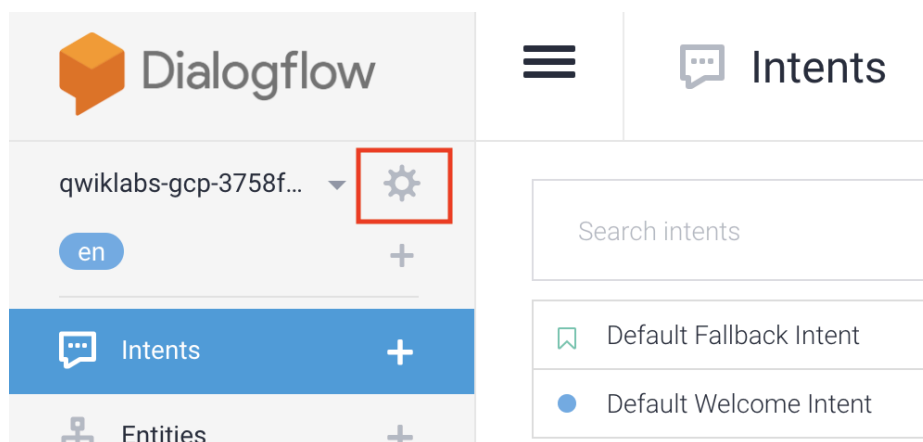
如要完成其他 Dialogflow 程式碼研究室，請略過本節，稍後再返回。

刪除 Dialogflow 代理程式

1. 按一下現有代理程式旁邊的



。



2. 在「一般設定」分頁中，向下捲動並點選「刪除這個代理程式」。
3. 在隨即顯示的視窗中輸入「刪除」，然後按一下「刪除」。

步驟 18 · 約 0 分鐘

18. 恭喜

您已在 Dialogflow 中建立聊天機器人，並與 Vision API 整合。你現在是聊天機器人開發人員！

瞭解詳情

如要瞭解詳情，請參閱 [Dialogflow Github](#) 頁面上的程式碼範例。
